Lab 3: Designing Memory Access Interface

Raymond Tran, Robert Bulanon, Anna Larson, Dhiab Vaheedudheen

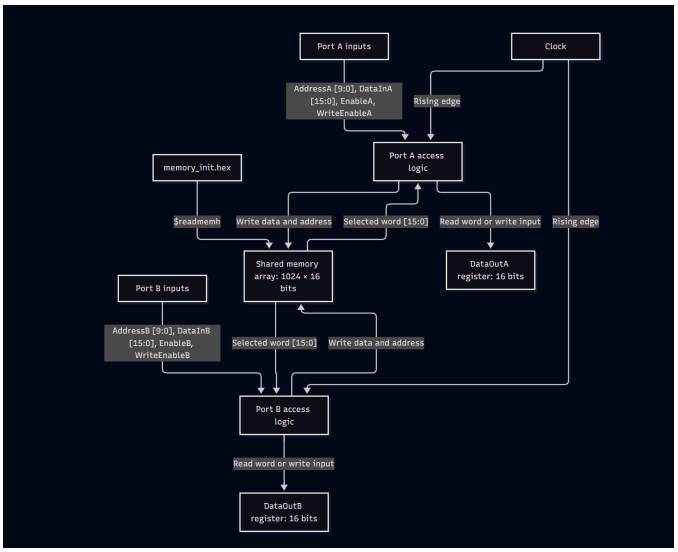


Fig. 1. Dual-port memory architecture showing independently controlled ports, a shared 1,024-word memory array, initialization, and registered outputs.

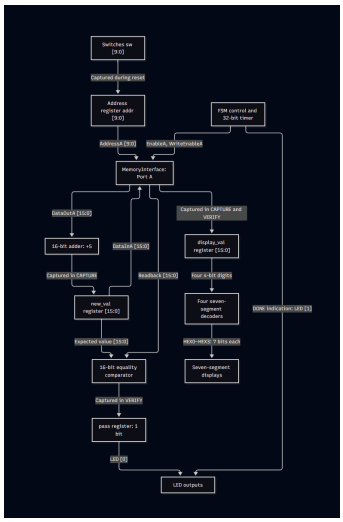


Fig. 2. Memory demonstration controller showing address selection, read-modify-write data flow, readback comparison, and display outputs.

I. Introduction

The objective of this lab is to design and test a synchronous dual-port memory interface in preparation for integration into the processor datapath. A demonstration FSM coordinates a read-modify-write-read sequence to verify memory operation. This report describes the memory organization, initialization, component connections, and controller behavior. Testbench simulations and a hardware demonstration are used to evaluate the design.

II. Architectural Additions and Accompanying Diagrams

The design consists of a synchronous dual-port memory module, MemoryInterface, and a demonstration controller, FSM_Mem_Interface_L3. The memory module provides two independent access ports to a shared array of 1,024 16-bit words. The controller performs a read-modify-write-read sequence at an address selected by the switches. It displays the original and updated values on four seven-segment displays and indicates whether verification succeeded using the LEDs. All sequential operations use the rising edge of the same clock. Figures 1 and 2 depict the dual-port memory module and FSM controller, respectively. Both will be expanded upon within the next sections.

III. Dual-port Memory Module

The MemoryInterface module contains a shared memory array declared as reg [15:0] Memory [0:1023]. Each port has a 10-bit address, a 16-bit data input, a 16-bit registered data output, an enable signal, and a write-enable signal. The 10-bit addresses select individual 16-bit words from locations 0 through 1023. An initial block uses `$readmemh` to load hexadecimal values from `memory_init.hex`.

Both ports operate synchronously. On a rising clock edge, an enabled port writes its input data to the selected address when its write-enable signal is high. The same input value is also assigned to that port's output register, giving the RTL write-first behavior for that port. When write-enable is low, the selected memory word is copied into the output register. When a port is disabled, its output retains its previous value, and that port performs no memory access.

The two ports share the same storage but have separate control signals and output registers. The module does not include resolution for simultaneous writes to the same address, so this condition must be avoided or addressed separately.

Figure 1 illustrates how the two access ports connect to the shared memory array. Each port's address selects a memory location, while its enable and write-enable signals determine whether that port performs a read, a write, or no operation. During a read, data flows from the selected memory location through the port's access logic into its output register. During a write, the input data flows into both the selected memory location and the corresponding output register. The separate access logic allows both ports to operate during the same clock cycle, provided conflicting accesses are avoided. The initialization path from `memory_init.hex` supplies the starting memory

contents, while the common clock synchronizes subsequent reads and writes.

IV. FSM/Controller Demonstration

The `FSM_Mem_Interface_L3` module connects the memory to the board switches, reset button, LEDs, and seven-segment displays. Reset is active low and synchronous: when `key` is low at a rising clock edge, the controller saves `sw[9:0]` as the memory address and resets its state, timer, data registers, and pass flag. The address remains unchanged until another reset.

Figure 2 shows how data moves through the controller. The address register connects to `AddressA`, while the FSM controls `EnableA` and `WriteEnableA`. The memory output connects to the display register, an adder, and a comparator. The adder adds five to the original value and stores the result in `new_val`. This register supplies the data written back to memory and the expected value used by the comparator.

The controller begins in `READ1`, which reads the selected address. On the following clock edge, `CAPTURE` saves the original value in `display_val` and the incremented value in `new_val`. Separate read and capture states are needed because the memory outputs are registered. After `SHOW_OLD` displays the original value, `WRITE` writes the updated value to the same address. Next, `READ2` rereads that address, and `VERIFY` compares the result with `new_val`, updates the pass flag, and saves the readback value for display.

A 32-bit timer holds each value on the displays for approximately two seconds at 50 MHz. Four seven-segment decoders convert the four 4-bit digits in `display_val` into outputs for `HEX0` through `HEX3`. A successful comparison turns on `LED[0]`. After `SHOW_NEW`, the controller enters `DONE`, turns on `LED[1]`, and waits for reset. The remaining LEDs stay off.

This demonstration uses only Port A. Port B is disabled by holding its enable and write-enable signals low, so simultaneous dual-port operation must be verified separately.